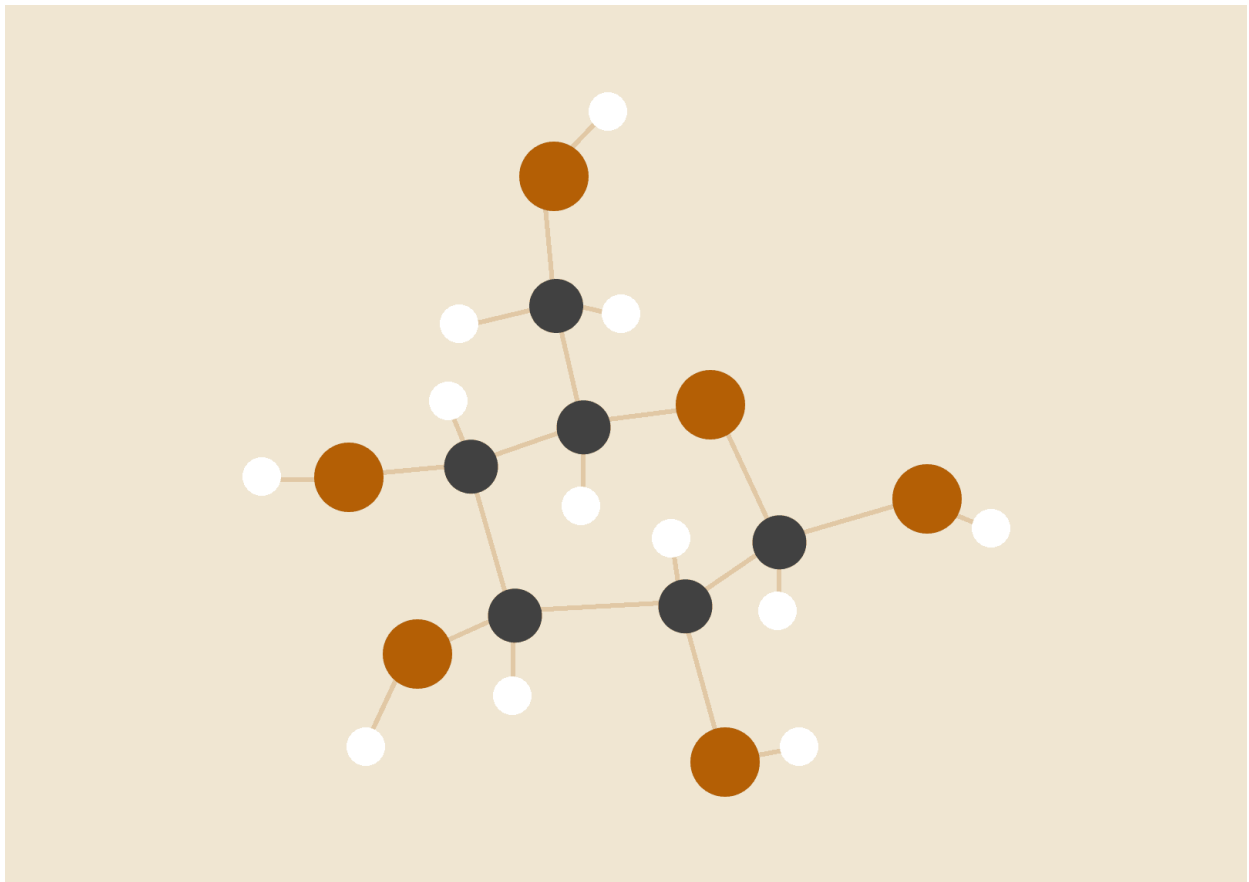


Data Engineering Project1 REPORT

Using *Snowflake* 

Kasi Mart Data Engineering



Motlalepula Lawrence Tshabalala

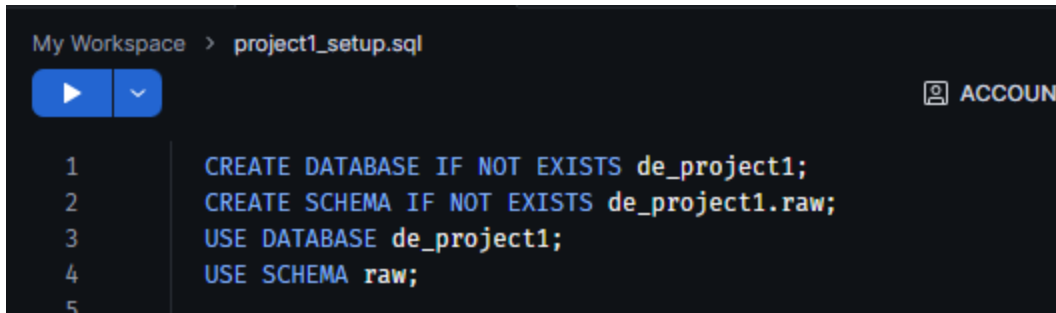
14.09.2026

1. Overview

This project loads three source files — `customers.csv`, `products.csv`, and `orders.csv` into a Snowflake database, confirms the load, and answers four business questions about revenue using SQL joins and aggregations.

2. Database & Schema Setup

Created a dedicated database and schema to isolate this project's objects:

A screenshot of a Snowflake SQL editor interface. The top bar shows 'My Workspace > project1_setup.sql'. Below the bar are two blue buttons: a play button and a dropdown arrow. On the right, there is a user icon and the text 'ACCOUNT'. The main area displays a SQL script with line numbers 1 through 5 on the left. The script consists of four lines: 'CREATE DATABASE IF NOT EXISTS de_project1;', 'CREATE SCHEMA IF NOT EXISTS de_project1.raw;', 'USE DATABASE de_project1;', and 'USE SCHEMA raw;'.

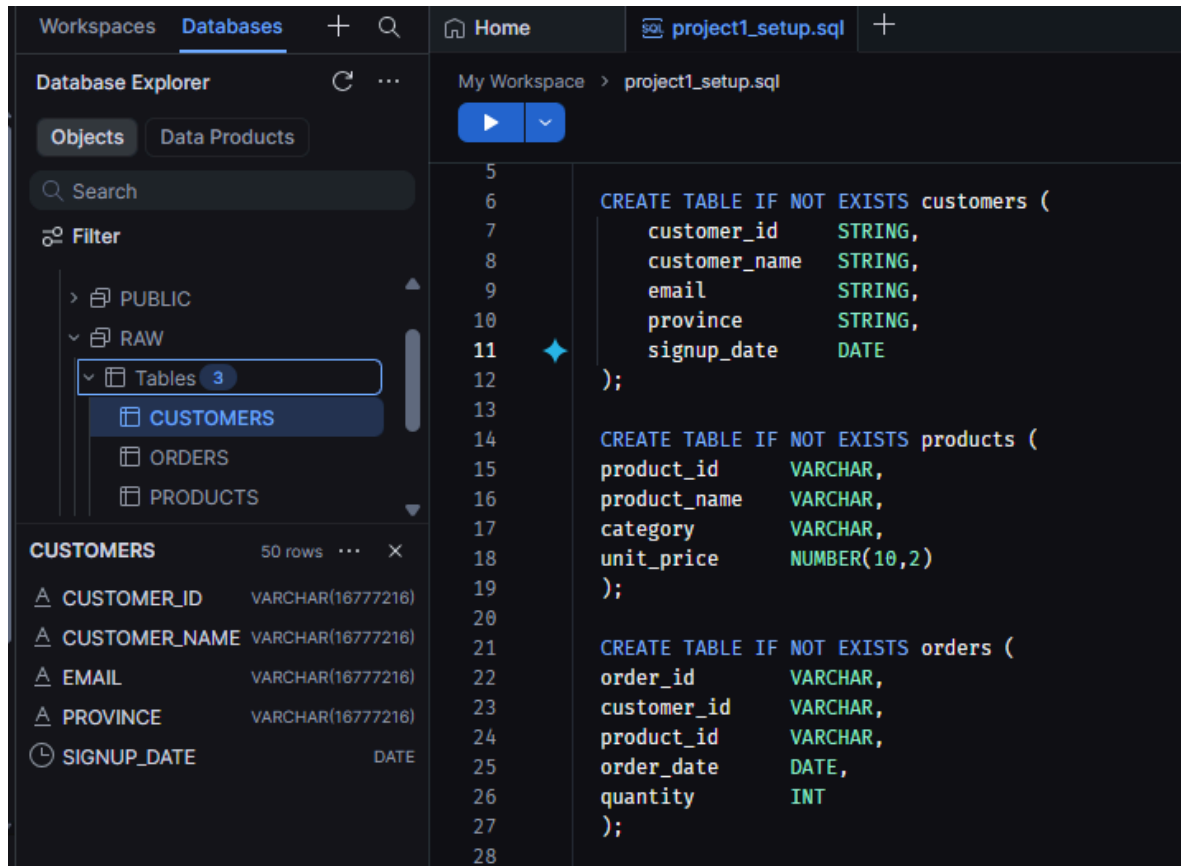
```
1 CREATE DATABASE IF NOT EXISTS de_project1;
2 CREATE SCHEMA IF NOT EXISTS de_project1.raw;
3 USE DATABASE de_project1;
4 USE SCHEMA raw;
5
```

Database: `DE_PROJECT1`

Schema: `RAW` (holds the raw, loaded source tables)

3. Table Creation (with correct data types)

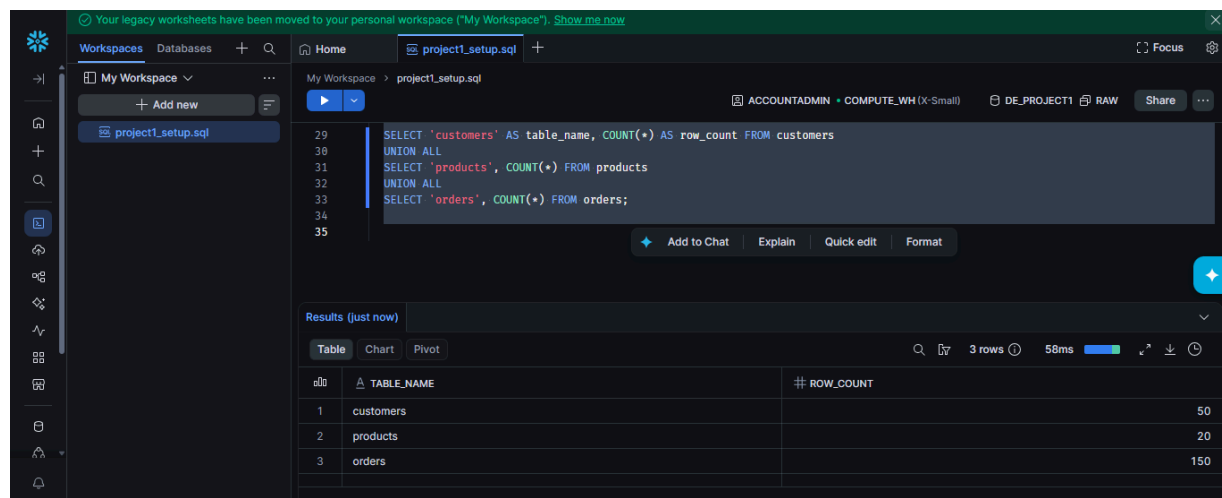
Each table was defined with types matching the data, rather than defaulting everything to text — text/IDs as `VARCHAR/STRING`, calendar values as `DATE`, whole counts as `INT`, and currency as `NUMBER(10,2)` for two-decimal precision.

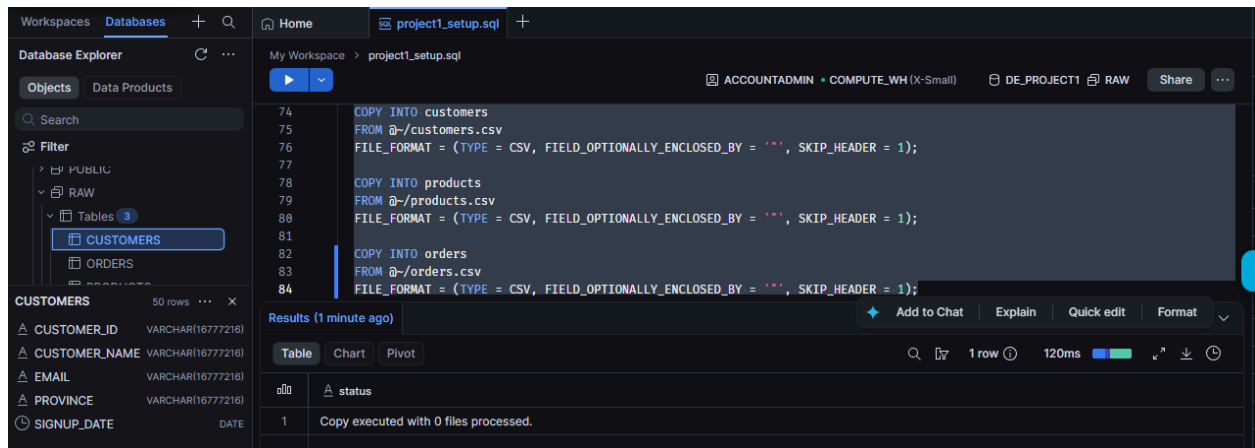


4. Data Load

Each CSV was loaded into its matching table using Snowflake's Load Data wizard, which uploads the file into a personal stage and runs an underlying **COPY INTO** load.

Load confirmation:





5. Queries

Query 1 — Order Detail Join

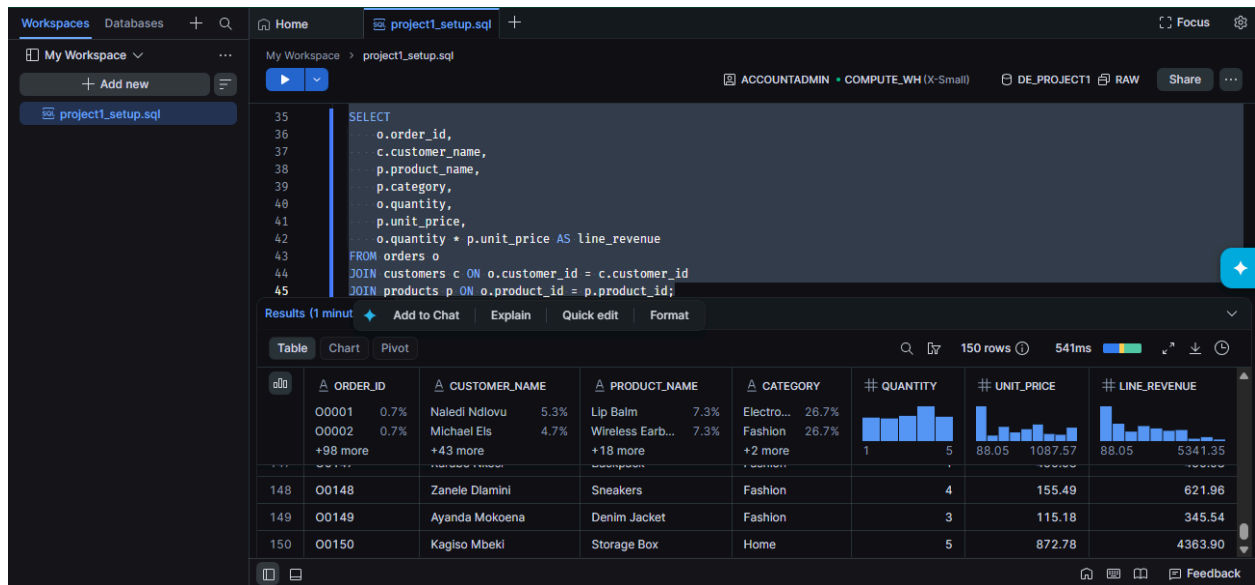
Every order joined to customer name, product name, category, and a calculated `line_revenue`.

```

35 SELECT
36     o.order_id,
37     c.customer_name,
38     p.product_name,
39     p.category,
40     o.quantity,
41     p.unit_price,
42     o.quantity * p.unit_price AS line_revenue
43 FROM orders o
44 JOIN customers c ON o.customer_id = c.customer_id
45 JOIN products p ON o.product_id = p.product_id;
46

```

Result: 150 rows — one per order, confirming every order matched a valid customer and product.



This query produces a row-level detailed view of every order placed at Kasi Mart, combining data from three separate tables into one readable record: who bought it, what they bought, what category it belongs to, and how much that single order line was worth. It matters because it's the foundation everything else is built on — without this granular, correctly joined detail, none of the summarized figures (revenue per customer, per category, or top spenders) could be calculated accurately. It also acts as a data quality check: getting exactly 150 rows back (matching the original order count) confirms every order successfully matched a real customer and product, with no orphaned or mismatched records.

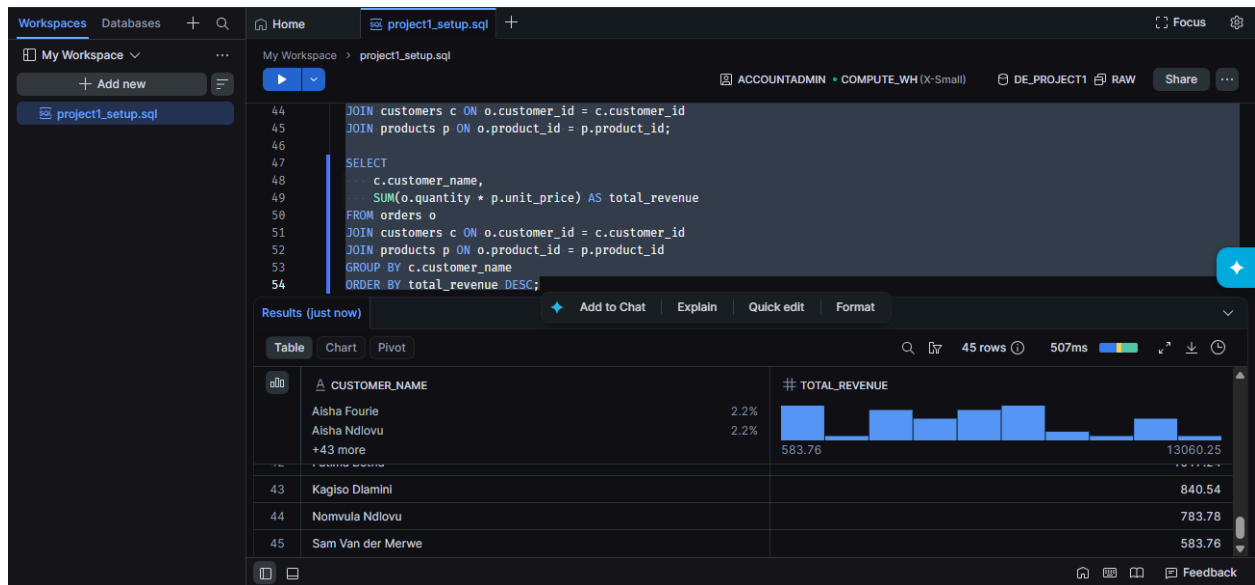
Query 2 — Total Revenue per Customer

```

46
47 SELECT
48     c.customer_name,
49     SUM(o.quantity * p.unit_price) AS total_revenue
50 FROM orders o
51 JOIN customers c ON o.customer_id = c.customer_id
52 JOIN products p ON o.product_id = p.product_id
53 GROUP BY c.customer_name
54 ORDER BY total_revenue DESC;
55

```

Result: 45 rows (customers with at least one order). An **INNER JOIN** was used deliberately, so customers who never placed an order are excluded from this view rather than shown with zero revenue.



This query aggregates every order per customer into a single total spend figure, sorted from highest to lowest. It matters because it turns transactional-level data into a customer-level view the business can actually act on — for example, identifying loyal or high-value customers for retention efforts. Note that this uses an inner join, so it only shows the 45 customers who have placed at least one order; the 5 customers with no orders are intentionally excluded rather than shown as zero, since the goal here was to rank active spend, not audit signup-to-purchase conversion.

Query 3 — Total Revenue per Product Category

```
56 SELECT
57     p.category,
58     SUM(o.quantity * p.unit_price) AS total_revenue
59 FROM orders o
60 JOIN products p ON o.product_id = p.product_id
61 GROUP BY p.category
62 ORDER BY total_revenue DESC;
```

Result:

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```

52 JOIN products p ON o.product_id = p.product_id
53 GROUP BY c.customer_name
54 ORDER BY total_revenue DESC;
55
56 SELECT
57     p.category,
58     SUM(o.quantity * p.unit_price) AS total_revenue
59 FROM orders o
60 JOIN products p ON o.product_id = p.product_id
61 GROUP BY p.category
62 ORDER BY total_revenue DESC;

```

The results pane shows the following table:

	CATEGORY	TOTAL_REVENUE
1	Beauty	87091.16
2	Home	72908.94
3	Fashion	49810.15
4	Electronics	47524.61

This query rolls up revenue by product category rather than by individual customer or order, showing which parts of the catalog drive the most sales. It matters for decisions like inventory planning, marketing spend, and supplier negotiations — for example, Beauty is the top-performing category here, which could justify expanding that range or double-checking stock levels. This query didn't need the customers table at all, since category-level revenue only depends on what was ordered (orders) and what it cost (products).

Query 4 — Top 5 Customers by Total Spend

```

63
64 SELECT
65     c.customer_name,
66     SUM(o.quantity * p.unit_price) AS total_revenue
67 FROM orders o
68 JOIN customers c ON o.customer_id = c.customer_id
69 JOIN products p ON o.product_id = p.product_id
70 GROUP BY c.customer_name
71 ORDER BY total_revenue DESC
72 LIMIT 5;
73

```

Result:

```

62 ORDER BY total_revenue DESC;
63
64 SELECT
65     c.customer_name,
66     SUM(o.quantity * p.unit_price) AS total_revenue
67 FROM orders o
68 JOIN customers c ON o.customer_id = c.customer_id
69 JOIN products p ON o.product_id = p.product_id
70 GROUP BY c.customer_name
71 ORDER BY total_revenue DESC
72 LIMIT 5;

```

	CUSTOMER_NAME	TOTAL_REVENUE
1	David Fourie	13060.25
2	Nomvula Coetzee	11721.14
3	Michael Els	11482.14
4	Karabo Nikosi	11004.24
5	Sipho Els	10908.90

This query narrows Query 2's full ranked list down to just the top 5 highest-spending customers. It matters because it gives the business a quick, actionable shortlist — these are the customers most worth prioritizing for loyalty rewards, personalized outreach, or churn prevention, since losing any one of them would have an outsized impact on revenue compared to an average customer.

6. CONCLUSION

This project walked through a complete, small-scale data engineering workflow in Snowflake: designing a schema with appropriate data types, loading raw CSV data into structured tables, validating that load with row counts, and then using SQL joins and aggregations to turn transactional order data into business-relevant insights. The four queries moved progressively from granular detail (every order line) to increasingly summarized views (per customer, per category, and a ranked top 5) — which mirrors how real analytics work usually flows: raw data first, then layers of aggregation on top to answer specific business questions.

The main technical lessons were around **data integrity and join behavior** — for example, understanding that `CREATE OR REPLACE TABLE` will silently wipe existing data if re-run, and that the choice between `INNER JOIN` and `LEFT JOIN` changes *which* records show up in a result (all customers vs. only those who transacted). Both are easy mistakes to make without noticing, and catching them here is good practice for larger, higher-stakes pipelines later on.

7. REFERENCES

1. Snowflake Documentation. *CREATE TABLE*.
<https://docs.snowflake.com/en/sql-reference/sql/create-table>
2. Snowflake Documentation. *Bulk loading from a local file system using COPY*.
<https://docs.snowflake.com/user-guide/data-load-internal-tutorial-copy-into>
3. Snowflake Documentation. *Data types overview*. <https://docs.snowflake.com>
4. Snowflake Web Interface introduction.
https://www.youtube.com/watch?v=PK-SbGX9_WM